

SIT720 - 8.1 DISTINCTION TASK (Machine Learning Mini Project) Report

Student ID - 226197828

Part 1

This project develops a housing price prediction and decision support model for a real estate agency treating sale price as the target variable to be estimated from measurable property and location attributes. Three Sydney suburbs were selected to represent genuinely different segments of the housing market accessible to a first home buyer, Mount Druitt and Campbelltown, established outer and south west suburbs offering standalone houses on larger blocks and Parramatta, a major employment and transport hub where affordability is achieved through unit rather than house ownership. The collected data illustrates why this mix matters. Median house prices reached \$1,021,250 in Mount Druitt and \$958,000 in Campbelltown, both above what a modest budget would typically allow, while Parramatta units carried a substantially lower median of \$589,000. This confirms that dwelling type, not just distance from the CBD is now a major determinant of affordability in Sydney and it justifies treating property type as a key explanatory feature.

There was significant variation in data quality between suburbs and housing types. Size of land area was unavailable for 86 percent of Parramatta houses as this was not applicable for strata title, while it was 20 to 25 percent in case of two house-dominated suburbs. Size of building or floor area was available for 6 out of 102 cases because usually no information about internal size is made available after selling. This is because any estimate would affect modeling later on. Another problem with sold listing data is that there is an element of selection bias involved. Listings which do not sell, get withdrawn or sell privately will be excluded from the dataset and will not form part of the sample.

```
#load the dataset
df = pd.read_csv("Sydney_Housing_Data.csv")

print("Total properties:", len(df))
print(df['Suburb'].value_counts())
✓ 0.0s
```

Total properties: 102	
Suburb	
Mount Druitt	36
Parramatta	36
Campbelltown	30
Name: count, dtype: int64	

```
#summary of the dataset
summary = df.groupby('Suburb')['Sale_Price_AUD'].agg(['count', 'median', 'min', 'max'])
print(summary)

missing_land = df.groupby('Suburb')['Land_Size_sqm'].apply(lambda x: round(x.isna().mean() * 100, 1))
print(missing_land)
```

✓ 0.0s

	count	median	min	max
Suburb				
Campbelltown	30	958000.0	620000.0	1750000.0
Mount Druitt	36	1021250.0	440000.0	1700000.0
Parramatta	36	589000.0	398000.0	800000.0
Suburb				
Campbelltown	20.0			
Mount Druitt	25.0			
Parramatta	86.1			

Name: Land_Size_sqm, dtype: float64

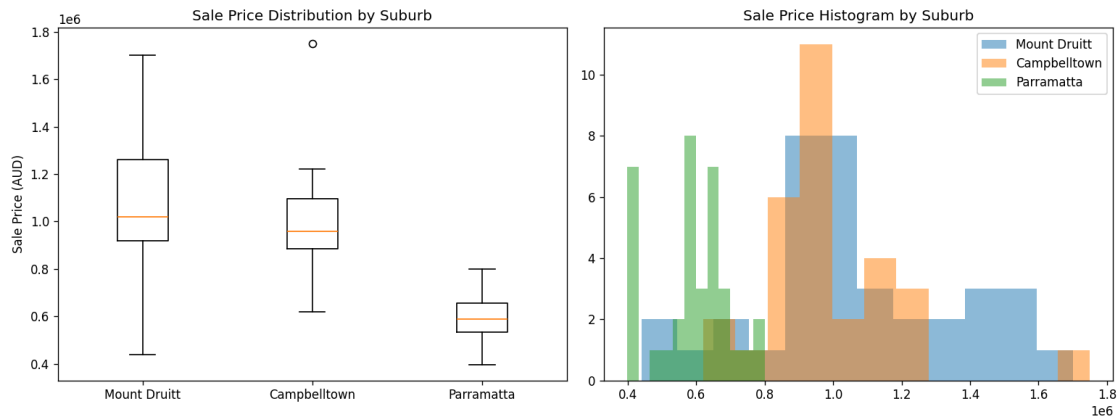
Part 2

Sale prices exhibit considerable separation between suburbs. Price medians of \$1,021,250 and \$958,000 were attained in Mount Druitt and Campbelltown respectively, significantly higher than Parramatta's \$589,000, with the boxplot indicating overlapping yet distinctly separated interquartile ranges, where the units sold in Parramatta make a more compact cluster below those in the two house-dominated suburbs. There was only one statistical outlier detected with the use of the 1.5 times IQR criterion, a seven-bedroom Campbelltown house sold at \$1,750,000 that may have been an unusually large family house rather than an erroneous price entry. On the other hand, interpreting the monthly price changes was complicated by Campbelltown's sale records only starting from June to August 2026, unlike Mount Druitt and Parramatta which had records starting from December 2025 to September 2026 and the apparent month-to-month fluctuations like that in Mount Druitt in June were due to low sample sizes per month.

```
df = pd.read_csv("Sydney_Housing_Data.csv")
df["Sale_Date"] = pd.to_datetime(df["Sale_Date"])

#price distribution and suburb comparison
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
order = ["Mount Druitt", "Campbelltown", "Parramatta"]
axes[0].boxplot([df[df.Suburb == s]["Sale_Price_AUD"] for s in order], tick_labels=order)
axes[0].set_title("Sale Price Distribution by Suburb")
axes[0].set_ylabel("Sale Price (AUD)")
for s in order:
    axes[1].hist(df[df.Suburb == s]["Sale_Price_AUD"], bins=12, alpha=0.5, label=s)
axes[1].set_title("Sale Price Histogram by Suburb")
axes[1].legend()
plt.tight_layout()
plt.savefig("price_distribution.png", dpi=120)
```

✓ 0.1s



Before engineering the features, it was anticipated that dwelling type, number of bedrooms and distance to CBD will be the three most significant variables for influencing price based on the general belief that property value is usually determined by land possession, space, and proximity to jobs. The dataset confirmed this to some extent. The number of bedrooms showed the highest correlation with price of 0.807, while comparing the houses with units resulted in a large difference in prices of \$1,030,145 and \$583,861. However, the correlation between distance to CBD and price turned out to be positive 0.624 instead of negative correlation expected which is caused by the fact that suburbs located farther away are dominated by the houses while the closest suburb, Parramatta, is dominated by units. In other words, in this case, distance is confounded with dwelling type. The two engineered variables, dwelling type indicator and total rooms showed the correlation of 0.697 and 0.729 correspondingly.

```
#outlier detection
print("Outliers")
for s in order:
    sub = df[df.Suburb == s]
    q1, q3 = sub.Sale_Price_AUD.quantile([0.25, 0.75])
    iqr = q3 - q1
    flagged = sub[(sub.Sale_Price_AUD < q1 - 1.5*iqr) | (sub.Sale_Price_AUD > q3 + 1.5*iqr)]
    for _, r in flagged.iterrows():
        print(f"{r.Property_ID} ({s}): {r.Property_Type}, {r.Bedrooms} bed, ${r.Sale_Price_AUD:,.0f}")
✓ 0.0s
```

Outliers
CB-015 (Campbelltown): House, 7 bed, \$1,750,000

```
#feature correlations with price
print("\nCorrelation with Sale_Price_AUD")
num_cols = ["Bedrooms", "Bathrooms", "Parking_Spaces", "Land_Size_sqm", "Distance_to_CBD_km"]
print(df[num_cols + ["Sale_Price_AUD"]].corr()["Sale_Price_AUD"].drop("Sale_Price_AUD").round(3))

#feature engineering
df["Dwelling_Group"] = df.Property_Type.apply(lambda x: 1 if x in ["House", "Villa", "Semi-detached", "Terrace"] else 0)
df["Total_Rooms"] = df.Bedrooms + df.Bathrooms
print("\nEngineered feature correlations")
print("Dwelling_Group (house=1):", round(df.Dwelling_Group.corr(df.Sale_Price_AUD), 3))
print("Total_Rooms:", round(df.Total_Rooms.corr(df.Sale_Price_AUD), 3))
print("\nMean price : House-like vs Unit:")
print(df.groupby(df.Dwelling_Group.map({1: "House-like", 0: "Unit"}))["Sale_Price_AUD"]
      .agg(["count", "mean"]).round(0))
```

✓ 0.0s

```
Correlation with Sale_Price_AUD
Bedrooms      0.807
Bathrooms      0.388
Parking_Spaces 0.553
Land_Size_sqm  0.681
Distance_to_CBD_km 0.624
Name: Sale_Price_AUD, dtype: float64

Engineered feature correlations
Dwelling_Group (house=1): 0.686
Total_Rooms: 0.729

Mean price : House-like vs Unit:
              count      mean
Dwelling_Group
House-like      65 1031147.0
Unit            37  594162.0
```

Part 3

Three models were selected to represent different points on the bias-variance curve, Linear Regression, as a simple parametric baseline assuming additive linear effects, a single Decision Tree, a flexible non-parametric model capable of capturing interactions but historically prone to overfitting on small samples and Random Forest, an ensemble of trees intended to reduce that overfitting through bagging while retaining non-linear flexibility.

Before training, Random Forest was expected to perform best given the non-linear suburb by dwelling type effects found in Part 2 with the single Decision Tree expected to overfit given only 102 properties and Linear Regression expected to remain a solid but slightly weaker baseline.

Five fold cross-validation broadly supported this. Random Forest achieved the lowest RMSE at \$152,988 and the highest mean R2 at 0.700, narrowly ahead of Linear Regression at RMSE \$160,149 and R2 0.694 while the Decision Tree performed worst on every metric with RMSE \$189,150 and R2 0.559. Comparing training R2 against cross-validated R2 exposed the story behind these averages. The Decision Tree fit training data almost perfectly at R2 0.979, but its cross-validated score collapsed to 0.559, a gap of 0.420 that is textbook overfitting for a dataset this small. Random Forest showed a smaller but still notable gap from 0.948 to 0.700, while Linear Regression generalised most consistently with only a 0.086 gap between training and cross-validated performance.

This changes the practical recommendation. While Random Forest was slightly better than Linear Regression on average, the variability of its R2 in cross-validation was extremely high with a standard deviation of 0.201, compared to Linear Regression where it was only 0.112, rendering its edge unreliable for this small sample size. Linear Regression should be used as the appropriate model for this task because its results are virtually the same as those of Random Forest, more robust across folds and easier to interpret for a decision support tool for a real estate agency.

```
df["Dwelling_Group"] = df.Property_Type.apply(lambda x: 1 if x in ["House", "Villa", "Semi-detached", "Terrace"] else 0)
df["Has_Land_Size"] = df.Land_Size_sqm.notna().astype(int)
house_medians = df[df.Dwelling_Group == 1].groupby("Suburb")["Land_Size_sqm"].median()

def fill_land(row):
    if pd.notna(row.Land_Size_sqm):
        return row.Land_Size_sqm
    if row.Dwelling_Group == 0:
        return 0.0
    return house_medians[row.Suburb]

df["Land_Size_sqm"] = df.apply(fill_land, axis=1)
df["Parking_Spaces"] = df["Parking_Spaces"].fillna(df["Parking_Spaces"].median())

#prepare another dataset with median estimates for filling gaps in the rows, keep this and raw data separate
df.to_csv("model_ready.csv", index=False)
print(df[["Bedrooms", "Bathrooms", "Parking_Spaces", "Land_Size_sqm", "Has_Land_Size", "Dwelling_Group"]].isna().sum())
✓ 0.0s

Bedrooms      0
Bathrooms      0
Parking_Spaces 0
Land_Size_sqm  0
Has_Land_Size  0
Dwelling_Group 0
dtype: int64
```

```
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import KFold, cross_validate

df = pd.read_csv("model_ready.csv")

X = pd.get_dummies(df[["Suburb"]], drop_first=True)
X = pd.concat([
    df[["Bedrooms", "Bathrooms", "Parking_Spaces", "Land_Size_sqm", "Has_Land_Size", "Dwelling_Group"]], X, axis=1)
y = df["Sale_Price_AUD"]

#three regression models
models = {
    "Linear Regression": LinearRegression(),
    "Decision Tree": DecisionTreeRegressor(random_state=42),
    "Random Forest": RandomForestRegressor(n_estimators=300, random_state=42),
}

#evaluate using k-fold cross validation
kf = KFold(n_splits=5, shuffle=True, random_state=42)
scoring = {"rmse": "neg_root_mean_squared_error", "mae": "neg_mean_absolute_error", "r2": "r2"}

for name, model in models.items():
    cv = cross_validate(model, X, y, cv=kf, scoring=scoring)
    print(name, "RMSE:", (-cv["test_rmse"]).mean(), "MAE:", (-cv["test_mae"]).mean(), "R2:", cv["test_r2"].mean())

    model.fit(X, y)
    train_r2 = model.score(X, y)
    print(f"  train R2 = {train_r2:.3f}, CV R2 = {cv['test_r2'].mean():.3f}, gap = {train_r2 - cv['test_r2'].mean():.3f}")
✓ 0.6s
```

```
Linear Regression RMSE: 160417.23286339152 MAE: 116555.62119558922 R2: 0.6934347515633377
  train R2 = 0.771, CV R2 = 0.693, gap = 0.077
Decision Tree RMSE: 192490.62800716804 MAE: 136272.7780013637 R2: 0.5467137023670247
  train R2 = 0.979, CV R2 = 0.547, gap = 0.432
Random Forest RMSE: 156857.11115663458 MAE: 118952.80803848481 R2: 0.681344660309068
  train R2 = 0.946, CV R2 = 0.681, gap = 0.265
```

Part 4

The five largest out-of-fold predictions generated from the recommended Linear Regression model involved houses in Mount Druitt or Campbelltown and none of the units in Parramatta. MD-011 is a four-bedroom house in Mount Druitt on the largest lot of the dataset which is 1120 square metres, this house was sold for \$1,000,000 while the prediction was \$1,491,139, which implies 49 percent overprediction because the model overweights the lot size. The other similar property is MD-001 which is a five-bedroom house on 815 square metres, but the predicted selling price of \$1,036,080 exceeds the actual price of \$732,275 by 41 percent, which can be explained by the same overgeneralisation. However, there were three underpredictions. First of all, CB-015, which is already defined as a statistical outlier in Part 2, then MD-031 is a small three-bedroom house with the selling price of \$1,325,000 and lastly, MD-023, which has 1012 square metres and this size corresponds to the values of Mount Druitt which have been defined as questionable in Part 2.

The shortcomings of the predictions illustrate the existence of other factors that are not included in the data such as the internal condition, time since renovation, the shape and subdivisibility of the block, and the exact location within the suburb, all of which are considered by real estate agents as additional factors determining prices besides simple counts and areas. Predictions have to be handled with care for bigger and more varied houses but can be made with more certainty for apartments, because all five largest errors are houses whereas the compactness of apartment prices in Parramatta yielded no such errors.

```
#investigate prediction failures
from sklearn.model_selection import KFold, cross_val_predict
df = pd.read_csv("model_ready.csv")

X = pd.get_dummies(df[["Suburb"]], drop_first=True)
X = pd.concat([
    df[["Bedrooms", "Bathrooms", "Parking_Spaces", "Land_Size_sqm", "Has_Land_Size", "Dwelling_Group"]], X], axis=1)
y = df["Sale_Price_AUD"]

kf = KFold(n_splits=5, shuffle=True, random_state=42)
df["Predicted_Price"] = cross_val_predict(LinearRegression(), X, y, cv=kf)
df["Error"] = df["Sale_Price_AUD"] - df["Predicted_Price"]
df["Pct_Error"] = (df["Error"] / df["Sale_Price_AUD"] * 100).round(1)

worst5 = df.reindex(df["Error"].abs().sort_values(ascending=False).index).head(5)
print(worst5[["Property_ID", "Suburb", "Property_Type", "Bedrooms", "Land_Size_sqm",
    "Sale_Price_AUD", "Predicted_Price", "Pct_Error"]].to_string(index=False))
```

✓ 0.0s

Property_ID	Suburb	Property_Type	Bedrooms	Land_Size_sqm	Sale_Price_AUD	Predicted_Price	Pct_Error
MD-031	Mount Druitt	House	3	325.0	1325000.0	8.562843e+05	35.4
CB-015	Campbelltown	House	7	510.0	1750000.0	1.284923e+06	26.6
MD-011	Mount Druitt	House	4	1120.0	1000000.0	1.431588e+06	-43.2
MD-023	Mount Druitt	House	3	1012.0	1485000.0	1.071946e+06	27.8
MD-001	Mount Druitt	House	5	543.0	900000.0	1.294114e+06	-43.8

Part 5

Property_ID	Suburb	Actual Price	ML Prediction	LLM Estimate	Human Estimate
MD-003	Mount Druitt	\$1,530,000	\$1,149,000 (24.9%)	\$1,080,000 (29.4%)	\$1,000,000 (34.6%)
PT-023	Parramatta	\$540,000	\$703,000 (30.2%)	\$680,000 (25.9%)	\$600,000 (11.1%)
MD-028	Mount Druitt	\$1,032,500	\$963,000 (6.7%)	\$1,000,000 (3.1%)	\$1,050,000 (1.7%)
PT-031	Parramatta	\$800,000	\$601,000 (24.9%)	\$650,000 (18.8%)	\$680,000 (15.0%)
MD-016	Mount Druitt	\$440,000	\$726,000 (65.0%)	\$650,000 (47.7%)	\$700,000 (59.1%)
PT-034	Parramatta	\$405,000	\$498,000 (23.0%)	\$480,000 (18.5%)	\$550,000 (35.8%)
MD-015	Mount Druitt	\$660,000	\$899,000 (36.2%)	\$720,000 (9.1%)	\$800,000 (21.2%)
CB-015	Campbelltown	\$1,750,000	\$1,305,000 (25.4%)	\$1,450,000 (17.1%)	\$1,380,000 (21.1%)
CB-020	Campbelltown	\$885,000	\$894,000 (1.0%)	\$950,000 (7.3%)	\$900,000 (1.7%)
CB-029	Campbelltown	\$880,000	\$944,000 (7.3%)	\$1,020,000 (15.9%)	\$1,050,000 (19.3%)

Across the ten held-out properties no single approach dominated. The LLM estimate achieved the lowest mean absolute error at \$162,250, human judgement was close behind at \$182,750 and the ML model trailed at \$194,850, though wins per property were nearly even at four for LLM and three each for ML and human. This suggests these approaches are complementary rather than one being straightforwardly superior, particularly given the small sample size of ten properties limits how much weight any single ranking should carry.

All three techniques performed equally badly in relation to MD-016, which was a two bedroom house at Mount Druitt, situated in 104 sqm area and costing only \$440,000, giving a difference of 65 percent, 48 percent and 59 percent respectively for ML, LLM and humans. As all three techniques were provided with similar basic features and have been estimated over the roof, there must be some common factor about which the methods are unaware of, possibly something about the internal condition or small size of the property that no amount of reasoning can compensate whereas

on the other hand, when applied to an average property in their own suburb, MD-028 and CB-020, they all came close enough to the actual prices with a difference of few percent points each.

In relation to the seven-bedroom outlier found in Campbelltown, CB-015, the LLM and the human estimation method had more advantage over the linear regression model, as the two were capable of justifying why the said property would demand an additional price beyond the linear estimate. In this way, it can be noted that human and LLM estimations are still valuable in making a forecast of property prices especially if the property is a typical or deviates from the data sets that the model has learned.

Part 6

The prediction tool was built as a Streamlit application, since Streamlit turns a trained scikit-learn model into an interactive web interface with minimal additional code, suiting a small decision-support prototype for a real estate agency. On startup, the app retrains the same Linear Regression pipeline used throughout this project directly from the cleaned dataset, rather than loading a separately saved model file, so predictions always reflect the current data rather than a version that can silently drift out of date.

A user selects a suburb and property type, enters bedrooms, bathrooms, parking and land size, and receives an estimated price along with the model's cross-validated accuracy and a caution about which property types are less reliably predicted, drawn directly from the Part 3 and Part 4 findings.

Sydney Housing Price Predictor

Decision-support tool for Mount Druitt, Campbelltown and Parramatta, trained on 102 manually collected sold properties. This gives an estimate to support a decision, it is not a formal valuation.

Suburb

Mount Druitt

Parking spaces

1

Property type

House

Land size (sqm) - enter 0 if not applicable or unknown

500

Bedrooms

3

Bathrooms

1

Predict sale price

Estimated sale price: \$905,071

Model: Linear Regression trained via 5-fold cross-validation

The key learning from this project was that data quality took up the bulk of time spent on this project, rather than modeling complexity. Verifying manually that there were one hundred sold property observations allowed us to identify accidental duplicates, generic references from sources and the same odd land size value that came to feature again later as the biggest error in our predictions in Part 4, which demonstrated that any early data issues carry right through to the behavior of our model in the end. Decisions about feature engineering, such as imputing different values for land size depending on whether the observation was for a unit or house was more influential to our model's accuracy than choosing between three possible algorithms, and the most impressive outcome of this project quantitatively was potentially the discovery that an explainable Linear Regression generalized better than a more complex Random Forest.

The most obvious trade-off was between predictive accuracy and deployability. Even though Random Forest had slightly better average cross-validated R^2 score, its instability in different folds and inability to provide meaningful coefficients made it less suitable as a tool to the real estate agent's judgment, compared to the Linear Regression Model which was proven to be inferior to both human and LLM reasoning in Part 5.

Ethically, this model reflects the biases of its training data. It was built entirely from properties that successfully sold and were publicly listed, so it cannot represent withdrawn sales, off market transactions or suburbs and property types outside the three sampled here and using it uncritically for a suburb like Mount Druitt could understate genuine variation tied to individual property condition rather than location. With more time, collecting internal condition and floor area consistently, extending the sample across a full year to remove the uneven date coverage identified in Part 2 and testing regularised linear models would likely improve both accuracy and fairness.

Project Github Link : <https://github.com/srikar117/8.1D-ML-Mini-Project>

References :

[1]Deakin University. (2026). Week 6 : Linear models. (Topics - 6.1 to 6.13)[Lecture Materials]. CloudDeakin.

[2]Deakin University. (2026). Week 8 : Nonlinear models (KNN and Decision Tree). (Topics - 8.1 to 8.14)[Lecture Materials]. CloudDeakin.

[3]Deakin University. (2026). Week 9 : Nonlinear models (Random Forest and Boosting) (Topics - 9.1 to 9.13)[Lecture Materials]. CloudDeakin.

[4]REA Group. (n.d.). realestate.com.au: Real estate to buy, rent & share in Australia. Retrieved September 2026. <https://www.realestate.com.au>

[5]Domain Group. (n.d.). Domain: Real estate for sale & rent in Australia. Retrieved September 2026. <https://www.domain.com.au>

[6]GenAI Acknowledgement: Claude (Anthropic, Sonnet 5) was used to assist with brainstorming, improving grammar, and explaining machine learning concepts. All code, analysis, interpretations, and final answers were independently reviewed, modified, and verified by the author before submission. This use complies with the assessment instructions.